CASE STUDY: SOCIAL NETWORK ADJACENCY MATRIX FOR REPRESENTING FRIENDSHIP CONNECTIONS AMONG MILLIONS OF USERS

1. Introduction

Social networking platforms such as Facebook, LinkedIn, and Instagram connect millions of users worldwide. Users can establish friendships, follow other users, and interact with their connections.

To represent these relationships mathematically, a graph data structure can be used. In a graph, users are represented as vertices (nodes), and friendship connections are represented as edges.

An **adjacency matrix** is a two-dimensional array used to represent the connections between vertices in a graph. It helps determine whether two users are directly connected.

2. Problem Statement

A social networking platform needs to represent friendship connections among millions of users efficiently.

The system must be able to:

- Represent users and their friendship relationships.
- Check whether two users are friends.
- Add or remove friendship connections.
- Analyze the relationships between users.

The challenge is to choose a suitable graph representation that balances access speed, memory usage, and scalability.

3. Objectives

- To understand graph representation using an adjacency matrix.
- To represent users as vertices and friendships as edges.
- To identify the advantages and limitations of adjacency matrices.
- To analyze time and space complexity.
- To understand why large social networks may require more memory-efficient representations.

4. Proposed System

The proposed system represents a social network as an undirected graph, assuming that friendship is mutual.

Let the graph be:

[
G=(V,E)
]

Where:

- (V) represents the set of users.
- (E) represents the set of friendship connections.

For (n) users, an adjacency matrix (A) is an (n \times n) matrix.

Each entry is defined as:

[
A[i][j]=
\begin{cases}
1 & \text{if users } i \text{ and } j \text{ are friends}\\
0 & \text{otherwise}
\end{cases}
]

Because friendship is mutual, the matrix is symmetric:

[
A[i][j]=A[j][i]
]

The diagonal entries are usually zero when users are not considered friends with themselves.

5. Example Dataset

Consider a small social network containing five users: Alice, Bob, Charlie, David, and Emma.

The friendship connections are:

- Alice is friends with Bob and Charlie.
- Bob is friends with Alice and David.
- Charlie is friends with Alice and David.
- David is friends with Bob, Charlie, and Emma.
- Emma is friends with David.

Adjacency Matrix

User	Alice	Bob	Charlie	David	Emma
Alice	0	1	1	0	0
Bob	1	0	0	1	0
Charlie	1	0	0	1	0
David	0	1	1	0	1
Emma	0	0	0	1	0

For example, the value at the intersection of Alice's row and Bob's column is 1, indicating that Alice and Bob are friends. The value at Alice's row and David's column is 0, indicating that they are not directly connected.

6. Algorithm

Algorithm to Check Friendship

1. Start.
2. Assign a unique ID to every user.
3. Create an adjacency matrix initialized with zeros.
4. When two users become friends, set the corresponding matrix entries to 1.
5. To check whether two users are friends, examine the matrix entry for their IDs.
6. If the entry is 1, return "Friends"; otherwise, return "Not Friends".
7. Stop.

Pseudocode

```
START

INPUT user1, user2

IF A[user1][user2] == 1
    PRINT "Users are friends"
ELSE
    PRINT "Users are not friends"

STOP
```

7. Python Implementation

```
# Social network using an adjacency matrix

users = ["Alice", "Bob", "Charlie", "David", "Emma"]
```

```
n = len(users)

# Create an n x n matrix filled with zeros
A = [[0] * n for _ in range(n)]

def add_friendship(user1, user2):
    i = users.index(user1)
    j = users.index(user2)

    A[i][j] = 1
    A[j][i] = 1  # Friendship is mutual

def check_friendship(user1, user2):
    i = users.index(user1)
    j = users.index(user2)

    if A[i][j] == 1:
        print(user1, "and", user2, "are friends.")
    else:
        print(user1, "and", user2, "are not friends.")

# Add friendship connections
add_friendship("Alice", "Bob")
add_friendship("Alice", "Charlie")
add_friendship("Bob", "David")
add_friendship("Charlie", "David")
add_friendship("David", "Emma")

# Check friendship
check_friendship("Alice", "Bob")
check_friendship("Alice", "David")

# Display the adjacency matrix
print("\nAdjacency Matrix:")
print("     ", users)

for i in range(n):
    print(users[i], A[i])
```

Sample Output

```
Alice and Bob are friends.
Alice and David are not friends.

Adjacency Matrix:
      ['Alice', 'Bob', 'Charlie', 'David', 'Emma']
Alice [0, 1, 1, 0, 0]
Bob [1, 0, 0, 1, 0]
Charlie [1, 0, 0, 1, 0]
David [0, 1, 1, 0, 1]
Emma [0, 0, 0, 1, 0]
```

8. Working of the System

The system assigns each user a unique index. Whenever a friendship is created, the matrix entries corresponding to both users are updated to 1.

For example, when Alice and Bob become friends:

```
A[Alice][Bob] = 1
A[Bob][Alice] = 1
```

The matrix can then be used to check direct friendship connections, identify a user's friends, and support basic graph operations.

9. Time and Space Complexity

Let (n) be the number of users and (m) the number of friendship connections.

Operat	Time Com
Check friendship	O(1)
Add friendship	O(1)
Remove friendship	O(1)
Initialize matrix	$O(n^2)$
Find all friends of one user	O(n)
Space required	$O(n^2)$

These complexities assume that user IDs map directly to matrix indices. The example Python program uses `users.index()`, which takes O(n) time, so its lookup operations are not strictly O(1). A user-to-index dictionary would avoid that extra lookup cost.

10. Memory Requirement for Millions of Users

An adjacency matrix requires (n^2) entries.

For a network with one million users:

[
n=1{,}000{,}000
]

[
n^2=10^{12}\text{ entries}
]

If each entry uses one byte, the matrix requires approximately 1 terabyte of storage, ignoring additional overhead. If each entry uses a typical four-byte integer, it would require approximately 4 terabytes.

This makes a conventional adjacency matrix impractical for a large social network where most users have only a small fraction of all possible friendships.

11. Advantages

- Simple and easy to understand.
- Provides constant-time friendship checks when users map directly to indices.
- Adding or removing a connection requires updating only two entries in an undirected graph.
- Suitable for small or dense networks.
- Useful for graph algorithms and mathematical analysis.

12. Limitations

- Requires $O(n^2)$ memory, even when very few users are connected.
- Initializing the matrix can be expensive for millions of users.
- Adding a new user to a conventional matrix may require resizing or restructuring it.
- Scanning a user's row takes O(n) time, even if that user has few friends.
- A standard matrix does not naturally store details such as friendship dates, statuses, or interaction histories.

13. Improved Approach for Millions of Users

For a large, sparse social network, an **adjacency list** or a compressed sparse representation is generally more memory-efficient than a conventional adjacency matrix.

An adjacency list stores only the connections that actually exist.

For example:

```
Alice   -> Bob, Charlie
Bob     -> Alice, David
Charlie -> Alice, David
David   -> Bob, Charlie, Emma
Emma    -> David
```

The adjacency-list representation typically requires O(n + m) space, where (m) is the number of edges. For an undirected graph, each friendship is normally stored twice, once for each user.

A real social networking platform may also use distributed databases, graph-processing systems, caching, and specialized indexing to handle large numbers of users and relationships.

14. Applications

- Representing friendships in social networks.
- Checking whether users are directly connected.
- Identifying mutual friends.
- Analyzing social network structures.
- Detecting communities and clusters.
- Supporting graph traversal and network analysis.

15. Conclusion

An adjacency matrix is a simple and effective way to represent friendship connections in a social network. It offers constant-time checks for direct relationships when users have direct matrix indices, making it suitable for small and dense graphs.

However, a matrix requires $O(n^2)$ memory, which becomes impractical for millions of users in a sparse network. In such cases, adjacency lists and other sparse graph representations are generally more scalable and memory-efficient.

Therefore, the choice of graph representation depends on the size of the network, the density of its connections, and the operations the application needs to perform.